Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 4: Support Ticket Priority Routing & Conditional Logic

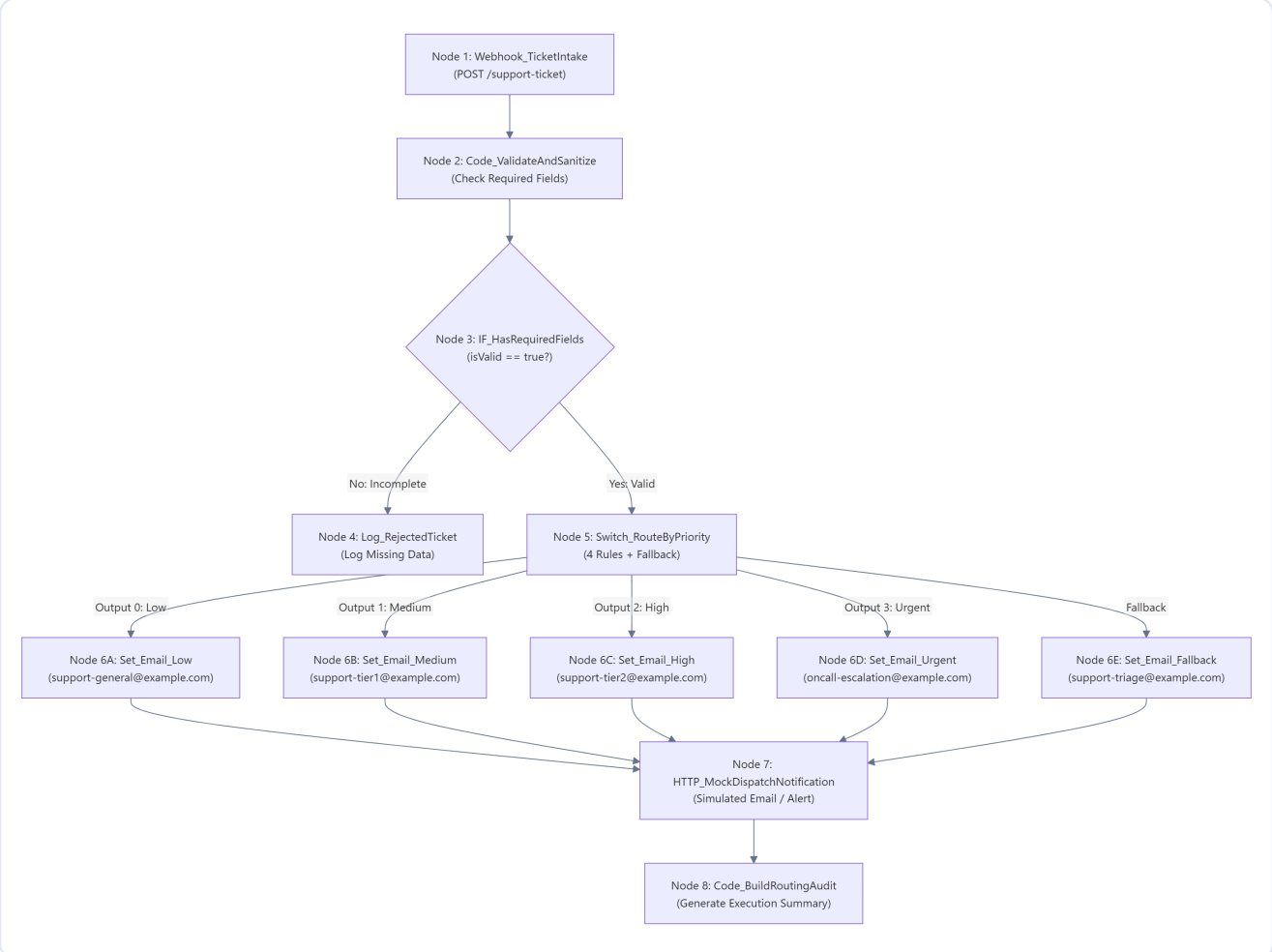
Document Navigation

Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an automated support ticket triage and routing workflow in n8n. The workflow ingests customer tickets via an HTTP webhook, validates required fields, routes items down 4 priority branches (Low, Medium, High, Urgent) plus an unrecognized Fallback branch via a Switch node, builds customized alert payloads, and dispatches them to a simulated support queue.

2. Canvas Architecture Overview



All rights reserved. Tayana Academy • Confidential Document

3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.
- [ ] Postman or cURL for dispatching test tickets.
- [ ] No external email service required (mock dispatcher included).

---

4. Step-by-Step Practical Build Guide

Step 1: Mock Notification Dispatcher Setup (Choose One of 3 Options)

To verify multi-tier ticket routing without a live SMTP mail server:

Option A: Free Cloud Mock API via DummyJSON Custom Response (Recommended)

1. Open **https://dummyjson.com/custom-response**.
2. In the JSON editor, paste:

```
{
  "dispatchId": "disp_dummy_44910",
  "status": "DELIVERED",
  "channel": "Simulated Support Gateway",
  "deliveredAt": "2026-10-15T12:00:00.000Z"
}
```

3. Set Status Code to `200 OK` and click **Generate API URL**.
4. Copy the generated URL (`https://dummyjson.com/c/your-id`) and paste into `HTTP_MockDispatchNotification`.

Option B: Postman Mock Server

1. In Postman, add a request `POST /notifications/dispatch`.
2. Add an Example with Status Code `200 OK` and response body:

```
{
  "dispatchId": "disp_pstmn_9921",
  "status": "DELIVERED",
  "gateway": "Postman Mock Mailer"
}
```

3. Click **Mock Collection** and copy the URL: `https://{{mock-id}}.mock.pstmn.io/notifications/dispatch`.
4. Paste into `HTTP_MockDispatchNotification`.

Option C: In-n8n Local Webhook Mock Server

All rights reserved. Tayana Academy • Confidential Document

1. Create a new workflow named `Mock Notification Server`.
2. Add a **Webhook Node** (`POST /mock/notifications/dispatch`, Mode: Using Respond Node).
3. Connect a **Respond to Webhook Node**:
   - **Response Code**: `200`
   - **Response Body (Expression)**:

```
{{
  {
    "dispatchId": "disp_" + Math.random().toString(36).substring(2, 9),
    "recipient": $json.body.recipientEmail,
    "subject": $json.body.subject,
    "routedQueue": $json.body.queue,
    "status": "DELIVERED",
    "sentAt": new Date().toISOString()
  }
}}
```

4. Save and activate the workflow. If running locally, your mock URL is: `http://localhost:5678/webhook/mock/notifications/dispatch` (or use Option A / Option B).

Step 2: Configure Ticket Ingestion Webhook (`Webhook_TicketIntake`)

1. Create a new workflow named `Topic 4 - Support Ticket Priority Routing`.
2. Add a **Webhook Node**:
   - **Node Name**: `Webhook_TicketIntake`
   - **HTTP Method**: `POST`
   - **Path**: `support-ticket`
   - **Response Mode**: `Immediately`
   - **Response Code**: `200`
   - **Response Data**:

```
{
  "status": "RECEIVED",
  "message": "Ticket queued for routing"
}
```

Step 3: Implement Validation & Field Normalization (`Code_ValidateAndSanitize`)

Connect a **Code Node** to verify mandatory parameters and normalize priority casing:

- **Node Name**: `Code_ValidateAndSanitize`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

All rights reserved. Tayana Academy • Confidential Document

```
const body = $input.item.json.body || $input.item.json;

const name = String(body.customerName || '').trim();
const email = String(body.customerEmail || '').trim().toLowerCase();
const subject = String(body.subject || '').trim();
const description = String(body.description || '').trim();
const rawPriority = String(body.priority || '').trim();

// Check presence of mandatory fields
const missing = [];
if (!name) missing.push("customerName");
if (!email) missing.push("customerEmail");
if (!subject) missing.push("subject");
if (!rawPriority) missing.push("priority");

const isValid = missing.length === 0;

// Normalize priority to Title Case (e.g. 'urgent' -> 'Urgent')
let normalizedPriority = "Fallback";
if (rawPriority) {
  normalizedPriority = rawPriority.charAt(0).toUpperCase() + rawPriority.slice(1).toLowerCase();
}

return {
  json: {
    isValid: isValid,
    missingFields: missing,
    ticket: {
      customerName: name,
      customerEmail: email,
      subject: subject,
      description: description,
      priority: normalizedPriority,
      rawPriority: rawPriority,
      submittedAt: new Date().toISOString()
    }
  }
};
```

Step 4: Add Validation Gate (`IF_HasRequiredFields`)

Add an **IF Node**:

- **Node Name**: `IF_HasRequiredFields`
- **Condition**: `{{ $json.isValid }}` equals `{{ true }}`

Connect the **False Branch** to a **Code Node** named `Log_RejectedTicket`:

```
return {
  json: {
    event: "TICKET_DROPPED_INCOMPLETE",
    missingFields: $json.missingFields,
    timestamp: new Date().toISOString()
  }
};
```

All rights reserved. Tayana Academy • Confidential Document

Step 5: Configure Multi-Branch Switch Node (`Switch_RouteByPriority`)

Connect the **True Branch** of `IF_HasRequiredFields` to a **Switch Node**:

- **Node Name**: `Switch_RouteByPriority`
- **Mode**: `Rules`
- **Value 1**: `{{ $json.ticket.priority }}`
- **Routing Rules**:
  - **Output 0**: Operation `Equal`, Value 2: `Low`
  - **Output 1**: Operation `Equal`, Value 2: `Medium`
  - **Output 2**: Operation `Equal`, Value 2: `High`
  - **Output 3**: Operation `Equal`, Value 2: `Urgent`
- **Fallback Output**: `Enabled` (Output 4 / Fallback)

Step 6: Configure the 5 Branch Payloads (Set Nodes)

Branch 0: Low Priority

Connect **Output 0** to a **Set Node** named `Set_Email_Low`:

- `recipientEmail`: `support-general@example.com`
- `queue`: `General Tier 0 Queue`
- `subject`: `New Ticket (Low): {{ $json.ticket.subject }}`
- `body`: `From: {{ $json.ticket.customerName }} ({{ $json.ticket.customerEmail }}) Issue: {{ $json.ticket.description }}`
- `priority`: `Low`

Branch 1: Medium Priority

Connect **Output 1** to a **Set Node** named `Set_Email_Medium`:

- `recipientEmail`: `support-tier1@example.com`
- `queue`: `Tier 1 Support Queue`
- `subject`: `New Ticket (Medium): {{ $json.ticket.subject }}`
- `body`: `From: {{ $json.ticket.customerName }} ({{ $json.ticket.customerEmail }}) Issue: {{ $json.ticket.description }}`
- `priority`: `Medium`

Branch 2: High Priority

Connect **Output 2** to a **Set Node** named `Set_Email_High`:

- `recipientEmail`: `support-tier2@example.com`
- `queue`: `Tier 2 Technical Escalations`

All rights reserved. Tayana Academy • Confidential Document

- `subject`: `High Priority Ticket: {{ $json.ticket.subject }}`
- `body`: `From: {{ $json.ticket.customerName }} ({{ $json.ticket.customerEmail }}) Issue: {{ $json.ticket.description }}`
- `priority`: `High`

Branch 3: Urgent Priority

Connect **Output 3** to a **Set Node** named `Set_Email_Urgent`:

- `recipientEmail`: `oncall-escalation@example.com`
- `queue`: `On-Call Urgent Page Desk`
- `subject`: `URGENT P1 ESCALATION: {{ $json.ticket.subject }}`
- `body`: `[IMMEDIATE ATTENTION REQUIRED] From: {{ $json.ticket.customerName }} ({{ $json.ticket.customerEmail }}) Issue: {{ $json.ticket.description }}`
- `priority`: `Urgent`

Fallback Branch: Unrecognized Priority

Connect the **Fallback Output** to a **Set Node** named `Set_Email_Fallback`:

- `recipientEmail`: `support-triage@example.com`
- `queue`: `Manual Triage Desk`
- `subject`: `Unrecognized Priority Review: {{ $json.ticket.subject }}`
- `body`: `Unclassified priority '{{ $json.ticket.rawPriority }}' detected. From: {{ $json.ticket.customerName }} ({{ $json.ticket.customerEmail }}) Issue: {{ $json.ticket.description }}`
- `priority`: `Fallback`

Step 7: Converge & Dispatch Notifications (`HTTP_MockDispatchNotification`)

Connect the outputs of all 5 Set nodes into a single **HTTP Request Node**:

- **Node Name**: `HTTP_MockDispatchNotification`
- **Method**: `POST`
- **URL**: `https://dummyjson.com/c/<your-id>` *(from Step 1 Option A, or your Postman Mock URL `https://{{mock-id}}.mock.pstmn.io/notifications/dispatch`)*
- **Send Body**: `ON`
- **Body Content Type**: `JSON`
- **Specify Body**: `Using JSON`
- **JSON**:

```
{{
  {
    "recipientEmail": $json.recipientEmail,
    "queue": $json.queue,
    "subject": $json.subject,
    "body": $json.body,
```

All rights reserved. Tayana Academy • Confidential Document

```
    "priority": $json.priority
  }
}}
```

Step 8: Build Routing Audit Record (`Code_BuildRoutingAudit`)

Connect to a **Code Node**:

- **Node Name**: `Code_BuildRoutingAudit`
- **Language**: `JavaScript`

```
const resp = $input.item.json;
return {
  json: {
    event: "TICKET_ROUTED_SUCCESSFULLY",
    dispatchId: resp.dispatchId,
    assignedQueue: resp.routedQueue,
    targetRecipient: resp.recipient,
    deliveryStatus: resp.status,
    timestamp: new Date().toISOString()
  }
};
```

---

5. Node-by-Node Settings Blueprint

Node ID	Node Name	Node Type	Critical Settings	Custom Expressions / Code
N4-01	`Webhook_TicketIntake`	Webhook	POST `/support-ticket`, Mode: Immediately, Code: 200	Acknowledge receipt
N4-02	`Code_ValidateAndSanitize`	Code	Check presence of 4 required fields, title-case priority	JavaScript schema validator
N4-03	`IF_HasRequiredFields`	IF	Mode: Basic, Condition: `isValid == true`	`{{ $json.isValid }} == true`
N4-04	`Log_RejectedTicket`	Code	Format audit log for dropped ticket	`event: "TICKET_DROPPED_INCOMPLETE"`
N4-05	`Switch_RouteByPriority`	Switch	Mode: Rules, 4 conditions + Fallback enabled	Rules: Low, Medium, High, Urgent
N4-06A	`Set_Email_Low`	Set	Recipient: `support-general@example.com`	`queue: "General Tier 0 Queue"`
N4-06B	`Set_Email_Medium`	Set	Recipient: `support-tier1@example.com`	`queue: "Tier 1 Support Queue"`
N4-06C	`Set_Email_High`	Set	Recipient: `support-tier2@example.com`	`queue: "Tier 2 Technical Escalations"`

All rights reserved. Tayana Academy • Confidential Document

N4-06D	`Set_Email_Urgent`	Set	Recipient: `oncall-escalation@example.com`	`queue: "On-Call Urgent Page Desk"`
N4-06E	`Set_Email_Fallback`	Set	Recipient: `support-triage@example.com`	`queue: "Manual Triage Desk"`
N4-07	`HTTP_MockDispatchNotification`	HTTP Request	POST to mock notification dispatcher	Dynamic dispatch JSON body
N4-08	`Code_BuildRoutingAudit`	Code	Format final execution summary	Audit summary object

---

6. Verification Checkpoints & Testing Guide

Test 1: Urgent Priority Ticket Escalation

Dispatch an Urgent ticket:

```
curl -X POST https://n8n.tayanasolutions.com/webhook/support-ticket \
  -H "Content-Type: application/json" \
  -d '{
    "customerName": "Alice Smith",
    "customerEmail": "alice@corporate.org",
    "subject": "Production Database Offline",
    "description": "Entire cluster reporting 502 Bad Gateway",
    "priority": "Urgent"
  }'
```

- **Expected Result**: Routes to `Set_Email_Urgent`. Mock dispatcher delivers to `oncall-escalation@example.com`.

Test 2: Low Priority Ticket Routing

Dispatch a Low priority ticket:

```
curl -X POST https://n8n.tayanasolutions.com/webhook/support-ticket \
  -H "Content-Type: application/json" \
  -d '{
    "customerName": "Bob Jones",
    "customerEmail": "bob@personal.io",
    "subject": "Typo in documentation",
    "description": "Page 4 has a misspelling",
    "priority": "low"
  }'
```

- **Expected Result**: Normalized to `"Low"`, routed to `Set_Email_Low`, dispatched to `support-general@example.com`.

Test 3: Unrecognized Priority Fallback Routing

Dispatch with priority `"Critical"` (not in standard 4 tiers):

```
curl -X POST https://n8n.tayanasolutions.com/webhook/support-ticket \
```

All rights reserved. Tayana Academy • Confidential Document

```
  -H "Content-Type: application/json" \
  -d '{
    "customerName": "Charlie Brown",
    "customerEmail": "charlie@peanuts.org",
    "subject": "Special Request",
    "description": "Custom billing question",
    "priority": "Critical"
  }'
```

- **Expected Result**: Matches no rule, routes to `Set_Email_Fallback`, delivered to `support-triage@example.com`.

Test 4: Missing Required Field Dropped

Dispatch payload missing `customerEmail`:

```
curl -X POST https://n8n.tayanasolutions.com/webhook/support-ticket \
  -H "Content-Type: application/json" \
  -d '{
    "customerName": "Anonymous",
    "subject": "Missing Email Test",
    "priority": "Medium"
  }'
```

- **Expected Result**: Rejected at `IF_HasRequiredFields`, logged in `Log_RejectedTicket`.

---

7. Production Risks & Mitigation Strategies

Risk	Impact	Mitigation
Case Sensitivity in Rules	Payload containing `"urgent"` bypasses rule and hits fallback.	Normalize priority with `.charAt(0).toUpperCase() + ...toLowerCase()` before Switch.
SMTP Server Outage	Live email node fails, crashing ticket processing.	Enable `Continue On Fail` or swap to an HTTP API email service (SendGrid, Postmark) with retry policies.
Unauthenticated Webhook Intake	Malicious users spam ticket endpoint.	Add Header Authentication or shared secret verification token.

All rights reserved. Tayana Academy • Confidential Document